

Qualidade e Teste de Software - JCRTQSW

Lista de Exercícios 01

As implementações pedidas nas questões a seguir podem ser feitas na linguagem de sua preferência.

- 1) Implementar uma classe ou função que implemente as regras de negócio de contratação de colaboradores descrita a seguir.

$0 \leq idade \leq 15$	Não empregar.
$16 \leq idade \leq 17$	Pode ser empregado tempo parcial.
$18 \leq idade \leq 54$	Pode ser empregado tempo integral.
$55 \leq idade \leq 99$	Não empregar.

- a. Derivar os casos de testes relativos ao Particionamento de Classes de Equivalência.
b. Derivar os casos testes relativos à Análise do Valor Limite.
c. Para os itens a) e b) acima, montar uma tabela para cada tipo de teste, contendo as colunas de dados de entrada (input do teste), resultado obtido (output do teste) e resultado esperado (oráculo). Lembrar que devem ser testados entradas válidas e inválidas.
- 2) Implementar uma classe ou função que implementa um algoritmo que verifica se uma determinada senha é forte, e que segundo a especificação ela deve ter pelo menos:
- 8 caracteres.
 - Uma letra maiúscula: ABCDEFG...
 - Uma letra minúscula: abcdefg...
 - Um número: 123456...
 - Um símbolo: @#\$.%
- a. Derivar os casos de testes relativos ao Particionamento de Classes de Equivalência.
b. Derivar os casos testes relativos à Análise do Valor Limite.
c. Para os itens a) e b) acima, montar uma tabela para cada tipo de teste, contendo as colunas de dados de entrada (input do teste), resultado obtido (output do teste) e resultado esperado (oráculo). Lembrar que devem ser testados entradas válidas e inválidas.

- 3) Implementar uma classe ou função que implementa um algoritmo que verifica se uma determinada senha é forte, conforme a especificação a seguir:
- I. A senha deve ter de 6 a 10 caracteres.
 - II. O primeiro caractere deve ser alfabético, numérico ou "?".
 - III. Os outros caracteres podem ser quaisquer, desde que não sejam caracteres de controle.
 - IV. A senha não pode existir num dicionário de palavras-chaves da disciplina de testes (por exemplo a palavra-chave teste ou funcional não pode ser usada).
- a. Derivar os casos de testes relativos ao Particionamento de Classes de Equivalência.
- b. Derivar os casos testes relativos à Análise do Valor Limite.
- c. Para os itens a) e b) acima, montar uma tabela para cada tipo de teste, contendo as colunas de dados de entrada (input do teste), resultado obtido (output do teste) e resultado esperado (oráculo). Lembrar que devem ser testados entradas válidas e inválidas.
- 4) Implementar uma classe ou função que implementa a interface bancária apresentada a seguir com a seguinte especificação:
- Código de área: vazio ou três dígitos numéricos que iniciem por "zero".
- Prefixo: três dígitos numéricos que não iniciem por "zero".
- Sufixo: quatro dígitos numéricos.
- Senha: seis dígitos alfanuméricos.
- Operação: c=cheque, d=depósito, e=extrato.
- a. Derivar os casos de testes relativos ao Particionamento de Classes de Equivalência.
- b. Derivar os casos testes relativos à Análise do Valor Limite.
- c. Para os itens a) e b) acima, montar uma tabela para cada tipo de teste, contendo as colunas de dados de entrada (input do teste), resultado obtido (output do teste) e resultado esperado (oráculo). Lembrar que devem ser testados entradas válidas e inválidas.

O diagrama mostra uma interface de usuário para uma aplicação bancária. No topo, há três campos de entrada para o código de área, prefixo e sufixo, com os valores 011, 344 e 2598 respectivamente. Abaixo, há um campo de entrada para a senha, com seis caracteres ocultos por asteriscos. No rodapé, há um campo de entrada para o comando, com o valor C. O nome do banco, 'banco', está escrito no canto inferior direito.

Cód. Área	Prefixo	Sufixo	Senha	Comando
011	344	2598	*****	C

banco